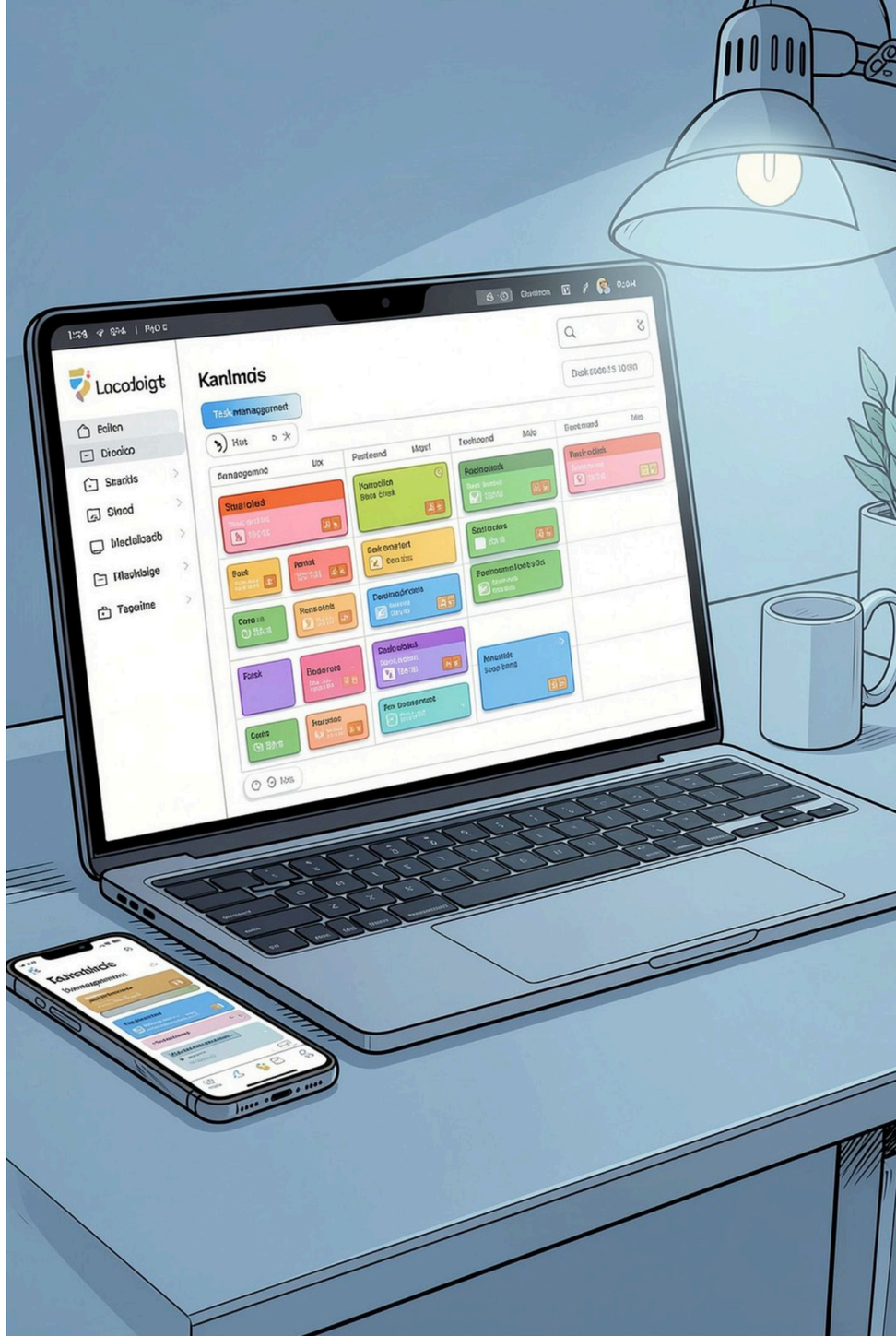
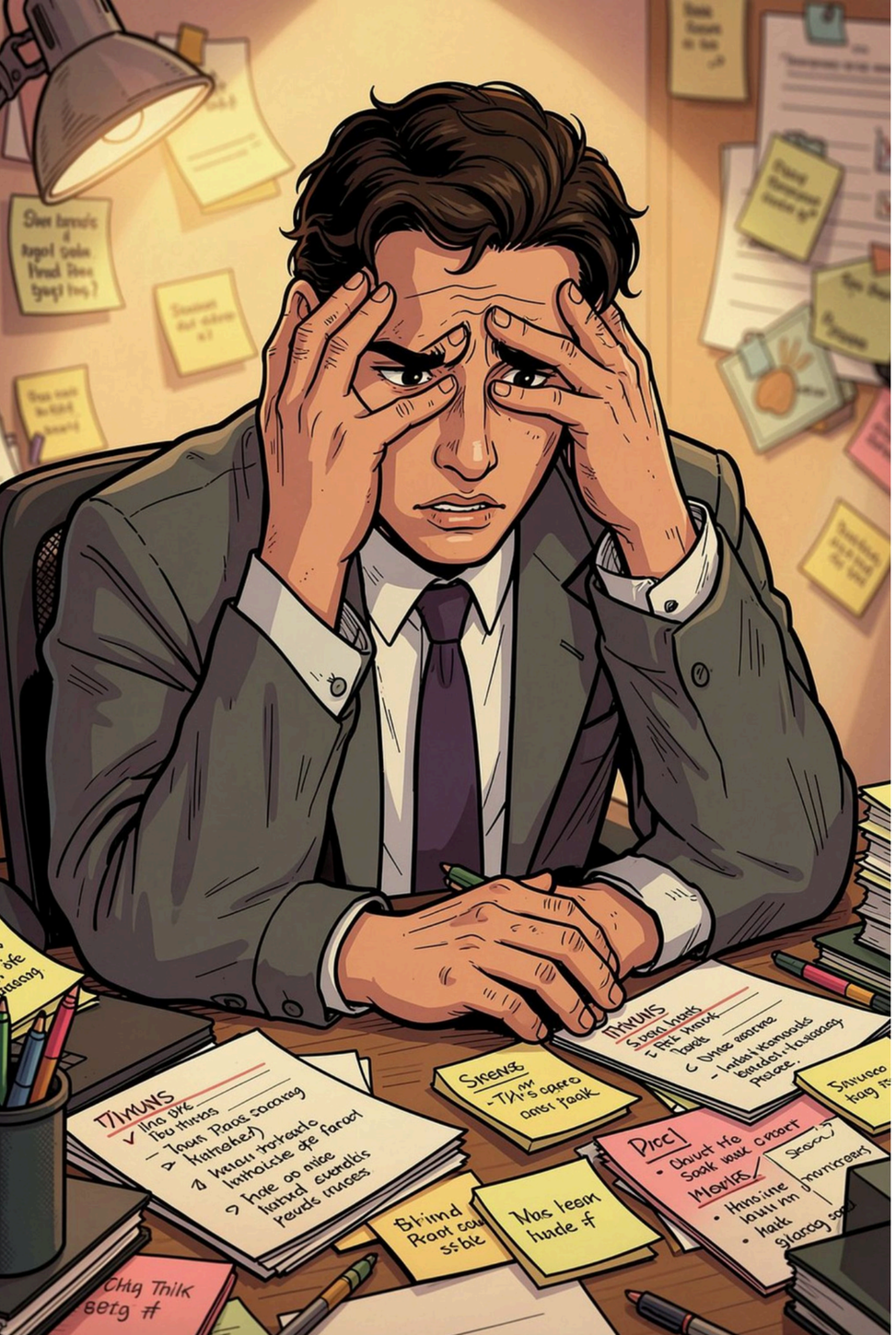
REVIEW 1 — SIMULATION USING MODERN TOOLS

Task Management System

Full-Stack Web Application — React + Node.js + MySQL

A responsive full-stack application for creating, organizing, tracking, and managing tasks efficiently.





PROBLEM STATEMENT

Why Task Management Matters

Manual Systems Fall Short

Paper and simple notes are difficult to update, reorganize, and maintain. Tasks are easily forgotten and priorities are hard to adjust.

Existing Tools Are Limited

Basic to-do apps lack flexible drag-and-drop organization, detailed status tracking, priority levels, and subtask support.

Data Persistence Is Critical

Without permanent storage, task information is lost between sessions — a critical gap for any real-world workflow.

INTRODUCTION

What the System Does

A full-stack web application designed to help users create, organize, and monitor tasks — with persistent storage and rich visualization.

Rich Task Details

Each task includes title, description, priority, category, due date, and subtasks.

Dual View Modes

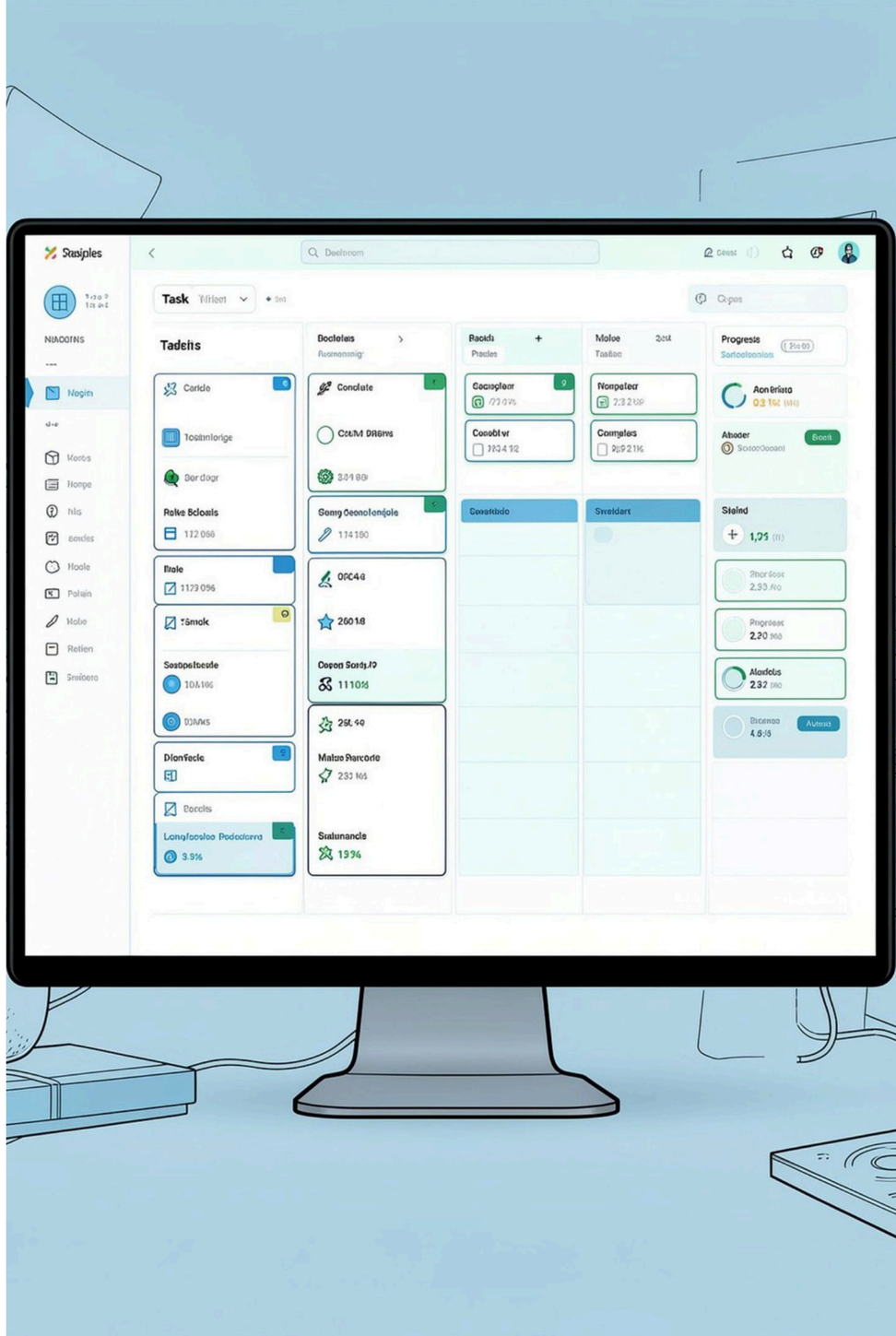
Tasks are visualized in both a structured List View and an interactive Kanban Board.

Persistent Storage

All task data is stored in MySQL, remaining available across sessions.

Search & Filtering

Locate tasks quickly using search and property-based filters.



MATHEMATICAL MODEL

Formal Task Representation

Task Tuple

Each task is modeled as a structured tuple:

$$T = \{I, N, D, S, P, C, \text{Due}, ST, O\}$$

- **I** — Task ID
- **N** — Name / Title
- **D** — Description
- **S** — Status
- **P** — Priority
- **C** — Category
- **Due** — Due Date
- **ST** — Subtasks
- **O** — Order

Derived Metrics

Completion Rate

$$\frac{\text{Completed}}{\text{Total}} \times 100$$

Subtask Progress

$$\frac{\text{Done Subtasks}}{\text{Total Subtasks}} \times 100$$

Task State Function

$$\text{State} = f(S, P, \text{Due}, ST, O)$$

MODELING TECHNIQUES

Core Techniques Applied

CRUD Operations

Create, retrieve, update, and delete task records through the REST API.

State Management

React state updates dynamically reflect every add, edit, or delete in real time.

Status Classification

Tasks are organized into **To Do**, **In Progress**, and **Completed** states.

Priority Organization

Tasks are categorized as Low, Medium, High, or Urgent for clear prioritization.

Drag-and-Drop Modeling

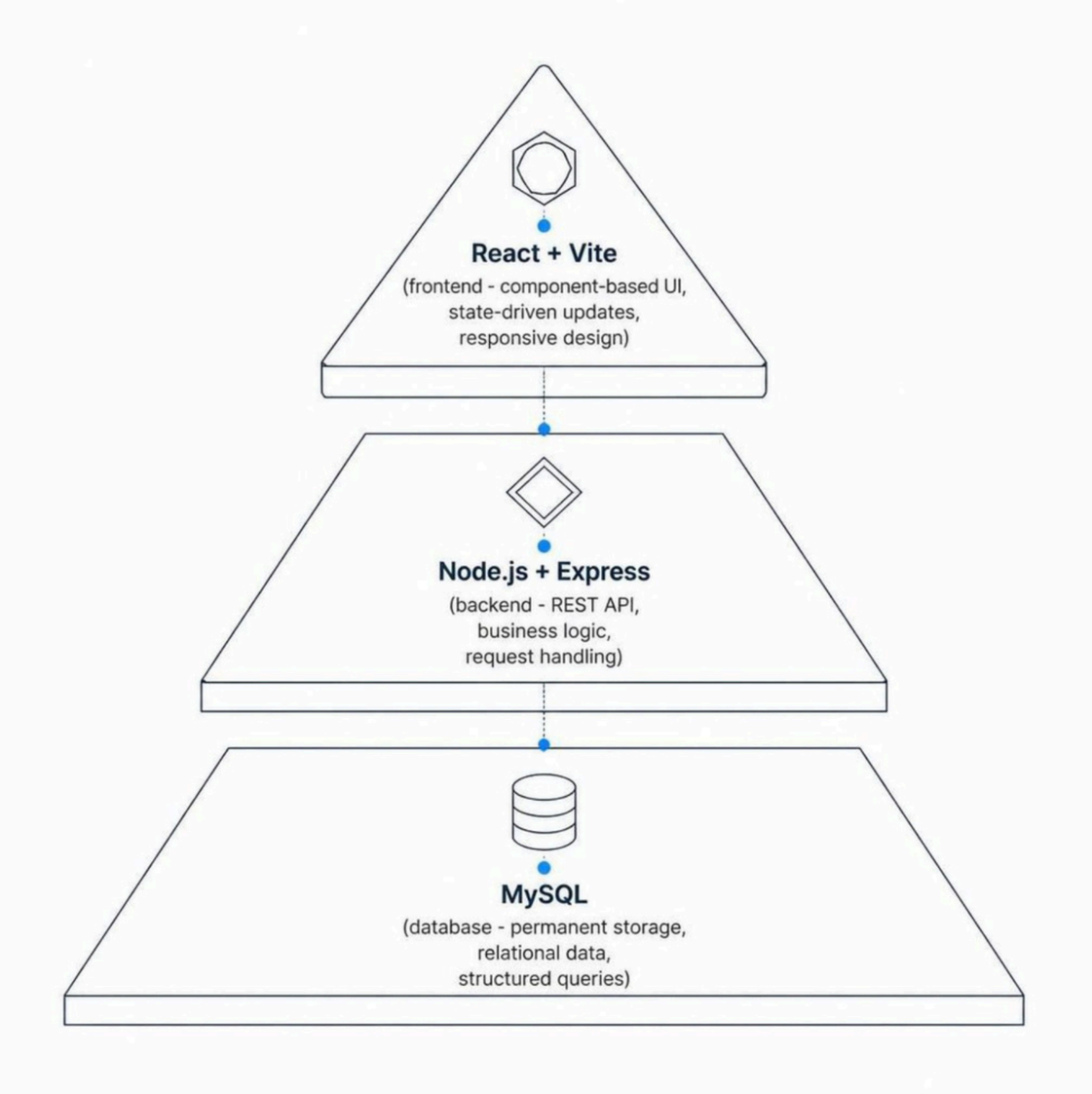
HTML5 Drag & Drop dynamically modifies task position and status within the Kanban board.

Progress Analysis

System calculates completed, pending, in-progress, and overdue task counts automatically.

TOOLS & TECHNOLOGIES

Technology Stack



Frontend

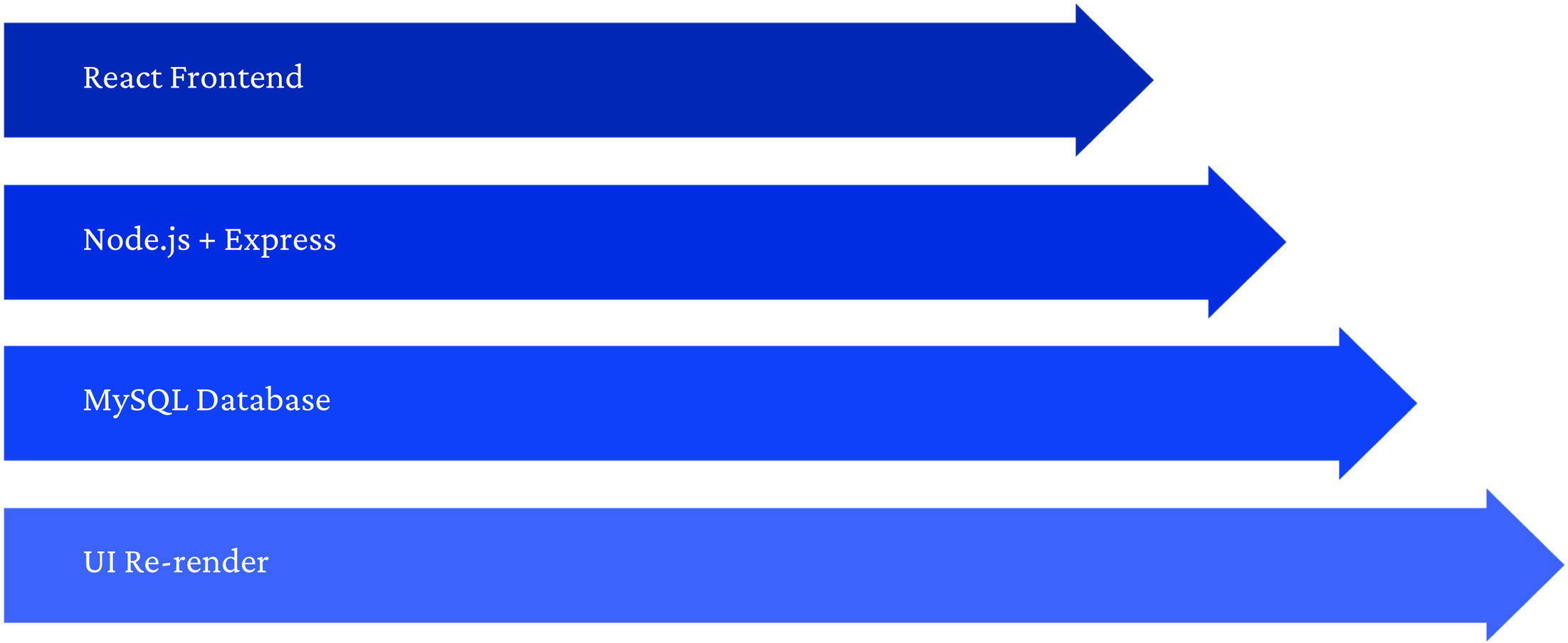
- **React + Vite** — Component-based, state-driven UI
- **HTML5 Drag & Drop** — Native task reordering
- **Lucide React** — Consistent icon set
- **Canvas Confetti** — Completion celebration
- **Web Audio API** — Interaction feedback

Backend & Infrastructure

- **Node.js + Express** — RESTful API layer
- **MySQL 8.0** — Persistent relational storage
- **Git / GitHub** — Version control

ARCHITECTURE & METHODOLOGY

System Architecture Flow



The methodology follows a clear five-stage cycle: **Input** (user creates or modifies tasks via React), **Process** (React sends API requests to Express), **Store** (backend validates and persists data in MySQL), **Retrieve** (tasks are fetched via REST endpoints), and **Visualize** (tasks render dynamically in List View, Kanban Board, and progress indicators).

Made with GAMMA

CURRENT IMPLEMENTATION

Core Modules — What's Built

Task Management

Create, edit, delete, and complete tasks with full property support.

Kanban Board

Visual workflow: **To Do → In Progress → Completed** with drag-and-drop transitions.

Subtask Management

Break larger tasks into smaller, trackable subtasks with independent completion states.

Search & Filter

Locate tasks by title, priority, category, or status instantly.

Analytics Dashboard

Live counts of Total, In Progress, Completed, and Overdue tasks.

Persistent Storage

All changes are committed to MySQL — data survives across sessions.

DATABASE DESIGN

MySQL Data Model

Field	Purpose
`id`	Unique task identifier
`title`	Task title
`description`	Task details
`completed`	Completion state (boolean)
`status`	To Do / In Progress / Completed
`priority`	Low / Medium / High / Urgent
`category`	Task category
`due_date`	Task deadline
`subtasks`	Stored subtask information
`order`	Task position for sorting
`created_at`	Creation timestamp
`updated_at`	Last update timestamp

Data Flow

- **User Action** → React State
- **REST API** → Express Controller
- **MySQL** → API Response
- **UI Re-render** with updated data

This schema extends the original 0th-review model (id, title, description, completed, order) with priority, category, due_date, subtasks, and timestamps.

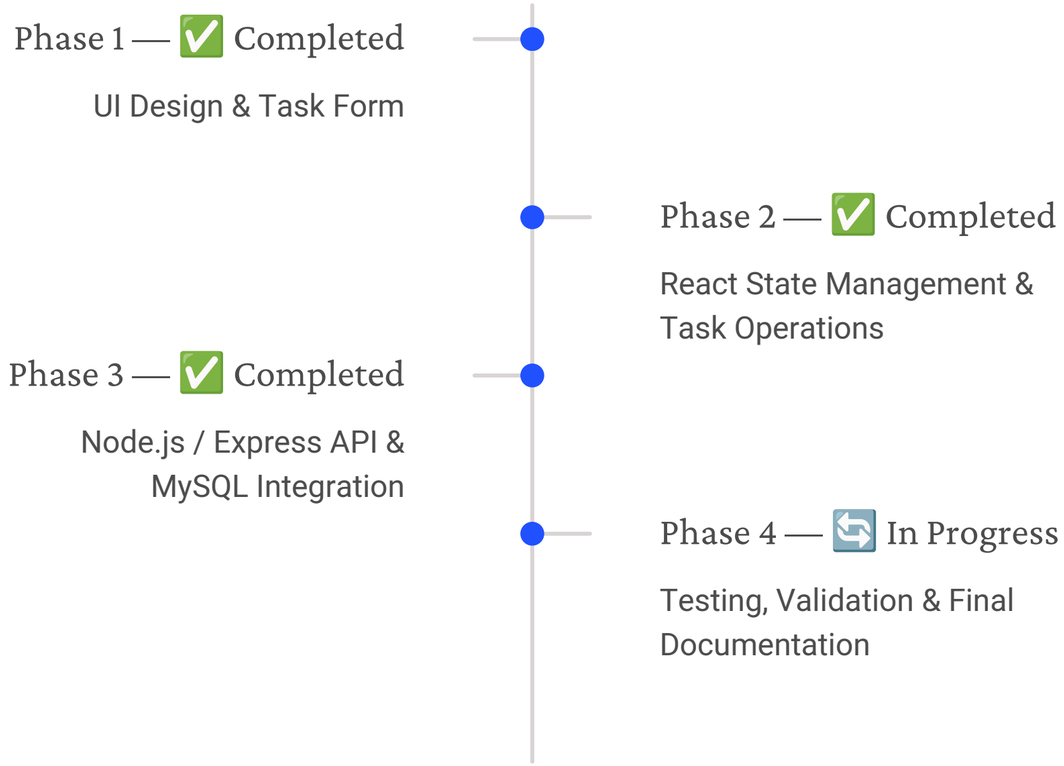
OUTCOMES & TIMELINE

Expected Outcomes & Project Status

Key Deliverables

- Complete full-stack task management application
- Interactive List View and Kanban Board
- Persistent MySQL storage across sessions
- Drag-and-drop task organization
- Task and subtask progress tracking
- Search, filters, and responsive design

Project Timeline



References: React, Node.js, Express.js, MySQL 8.0, MDN Web Docs (HTML5 Drag & Drop), Vite, GitHub Documentation.